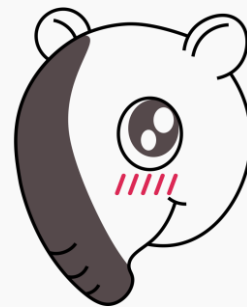


Vue d'ensemble de Tapir



Advanced Programming Paradigms
Eva Ray

1er Juin 2026
Lausanne
HES-SO MSE

Table des Matières

01

Présentation
de Tapir

02

Cas
d'utilisation

03

Forces et
Faiblesses

04

Démo

05

Conclusion

01

Présentation de Tapir

Qu'est-ce que c'est et à quoi ça sert?

Introduction

Tapir est une *bibliothèque* Scala qui permet de décrire des *APIs HTTP*, de les exposer en tant que *serveur*, de les consommer en tant que *client* et de les *documenter automatiquement* en utilisant des standards classiques.

- Toutes les implémentations populaires de serveur HTTP en Scala sont supportées (http4s, pekko, akka, netty, etc.). Tapir s'intègre à ces stacks.
- L'API entière peut être définie en utilisant Tapir mais il est aussi possible d'exposer des routes gérées par Tapir à côté d'autres routes «natives» (utile pour une adoption graduelle).
- Tapir fournit des intégrations avec de nombreuses bibliothèques de type personnalisé, JSON et d'observabilité.
- La documentation peut être générée aux formats: OpenAPI, AsyncAPI et JSON Schema.

Endpoints

- Tapir profite au maximum du typage fort proposé par Scala.
- Les endpoints sont fortement typés et séparent la description du endpoint (le «quoi») et sa logique (le «comment»).
- Un endpoint est une valeur du type **Endpoint[A, I, E, O, R]**:
 - **A**: type des paramètres d'entrée liés à la sécurité
 - **I**: type des paramètres d'entrée
 - **E**: type des paramètres de sortie en cas d'erreur
 - **O**: type des paramètres de sortie en cas de succès
 - **R**: capacités requises par les entrées/sorties de l'endpoint, comme le support des WebSockets ou d'une implémentation de streaming non-bloquante (Any si besoin de rien).

Endpoints

- On peut voir un endpoint comme une fonction qui prend des paramètres de type `A` et `I` et qui retourne un résultat du type `Either[E, O]`.
- Un endpoint sans entrées liées à la sécurité (`A`) s'appelle un **PublicEndpoint**`[I, E, O, -R]`.

```
val getItemEndpoint: Endpoint[Unit, Int, ApiError, Item, Any] =  
  endpoint  
    .get // méthode HTTP  
    .in("items" / path[Int]("id")) // chemin + type de l'input  
    .errorOut(jsonBody[ApiError]) // type de l'erreur  
    .out(jsonBody[Item]) // type de la réponse en cas de succès
```

Server Endpoints

- Pour interpréter un endpoint comme un serveur, il faut coupler sa description avec des fonctions qui implémentent la logique du serveur.
- La combinaison endpoint + logique a le type **ServerEndpoint[R, F]**, où R sont les requirements (websockets, streams) et F le type d'effet de la logique (IO ou Future)
- Pour les PublicEndpoint, on peut ajouter la logique grâce à la méthode: **serverLogic(f: I => F[Either[E, O]])**
- Le type de retour Either rend le cas d'erreur explicite (warning si pas traité)

```
val getItemLogic: ServerEndpoint[Any, IO]=  
  getItemEndpoint  
  .serverLogic { id =>  
    db.find(id).map {  
      case Some(u) => Right(u)  
      case None    => Left(NotFound(id))  
    }  
  }
```

Comparaison avec http4s

http4s

```
def routes: HttpRoutes[IO] =
  HttpRoutes.of[IO] {

    case GET -> Root / "users" / id =>
      val userId = id.toInt // erreur au
                           // runtime si «abc»

      db.find(userId).flatMap {
        case Some(u) => Ok(u.asJson)
        // None pas géré -> réponse vide
      }

    case POST -> Root / "users" =>
      req.as[User].flatMap { user =>
        db.insert(user).flatMap(_ => Created())
      }
  }
```

Tapir

```
val getUser = endpoint
  .get
  .in("users" / path[Int]("id"))
  .errorOut(jsonBody[NotFound])
  .out(jsonBody[User]) // type garanti

val createUser = endpoint
  .post
  .in("users")
  .in(jsonBody[CreateUserRequest])
  .errorOut(jsonBody[ValidationError])
  .out(jsonBody[User])
```

```
val getUserLogic =
  getUser.serverLogic { id => n// id: Int garanti
    db.find(id).map {
      case Some(u) => Right(u)
      case None    => Left(NotFound(id))
      // cas d'erreur explicite
    }
  }
```

description

logique

02

Cas d'utilisation

Cas d'utilisation

- **APIs publiques avec documentation:** Exposition d'une spec OpenAPI à des clients externes → La doc est toujours à jour par construction.
- **Domaines métier complexes:** Quand le modèle de données change souvent, le compilateur te liste exactement ce qui casse à chaque changement → impossible d'oublier un endpoint.
- **Architecture microservices:** Le contrat est partagé à la compilation entre serveur et clients → impossible de désynchroniser.
- **Streaming de données:** SSE, WebSockets, ou gros fichiers en chunks → la complexité bas niveau (le flux) est gérée par l'interpréteur, le flux est décrit de manière déclarative (juste ce que l'endpoint doit produire, pas comment).

Utilisateurs de Tapir



03

Forces et Faiblesses

Avantages

Type-Safety

Garanties à la compilation,
autocomplétion pendant le
développement

Déclaratif

Sépare la forme de l'endpoint
(le "quoi") de la logique
serveur (le "comment")

Documentation Générée

Génère la documentation
depuis les descriptions
d'endpoints

Observabilité

Exploite les métadonnées
pour exposer des
métriques et des
informations de tracing

Abstraction

Réutilise des définitions
d'endpoints communes, ainsi
que des entrées/sorties
individuelles

Pas un framework

S'intègre à n'importe quel
stack existant car c'est une
bibliothèque (supporte
énormément de bibliothèques)

Désavantages

Learning Curve

Demande d'être familier avec les concepts de programmation fonctionnelle et de typage avancé

Complexité

Ajoute beaucoup de complexité pour des APIs simples

Gain asymétrique

Tapir brille plus dans un contexte Scala full stack, avec un client qui utilise aussi Tapir

Pas un framework

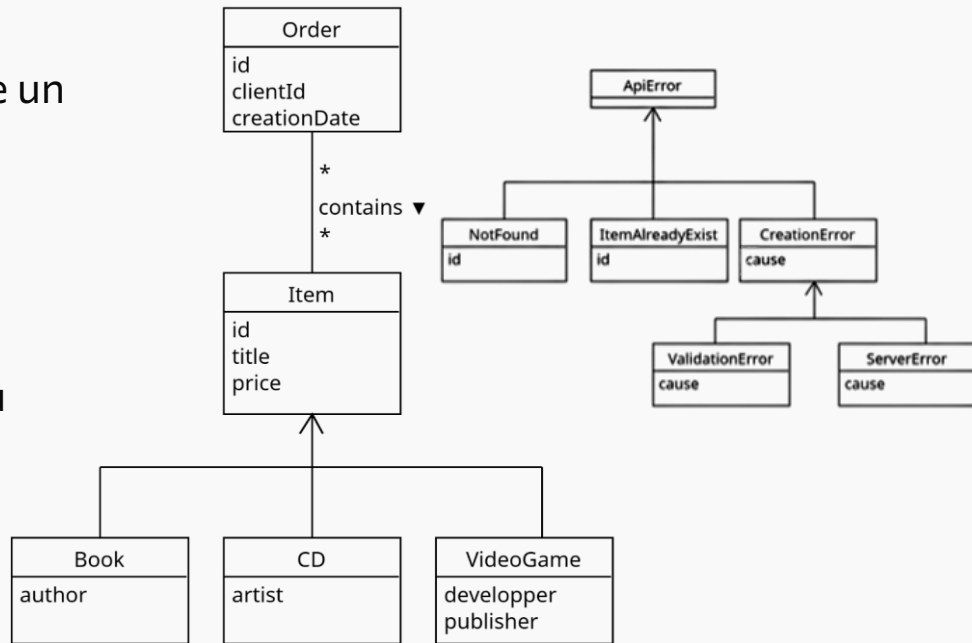
Tapir ne gère pas la DB, l'auth, la configuration, il faut assembler sa propre stack

04

Démonstration

Démonstration

- Application client/serveur qui simule un magasin multimédia
- **Endpoints:**
 - GET /items
 - GET /items/id
 - POST /items/books ou cds ou videogames
 - GET /orders
 - GET /orders/id
 - POST /orders
- **Technos:** Tapir, http4s, sttp, Circe, Swagger



05

Conclusion

À Retenir

- Tapir est une **librairie**, pas un framework → elle s'intègre au stack existant sans le remplacer
- Un endpoint est une **valeur typée** → la description, la logique, la doc et le potentiel client en dérivent tous
- Les erreurs de contrat sont détectées à la **compilation**, pas en production
- Le bénéfice est **maximal en full-stack Scala**, mais la doc OpenAPI auto-générée et les **ServerEndpoint** apportent déjà de la valeur
- Il existe une **courbe d'apprentissage**: il faut savoir gérer les types complexes.

Quand choisir Tapir?

Lorsqu'on veut qu'une API soit un contrat vérifié par le compilateur plutôt qu'une convention documentée à la main

Merci!

Avez-vous des questions?

Références

- [1] SoftwareMill. *Tapir documentation*. <https://tapir.softwaremill.com/en/latest/>
- [2] SoftwareMill. *Tapir – GitHub*. <https://github.com/softwaremill/tapir>
- [3] Baeldung. *Introduction to Tapir in Scala*. <https://www.baeldung.com/scala/tapir>
- [4] Lovindata. *Master Scala REST APIs in 3 Simple Concepts*.
<https://lovindata.github.io/2024/02/21/master-scala-rest-apis-in-3-simple-concepts-illustrated-guide-with-tapir-http4s-and-circe/>
- [5] Fiore A. *ACLs in Tapir*. <https://afiore.github.io/website/blog/2020/11/02/acls-in-tapir>